

ASS 3

Basics of machine learning:

Write the answers to the following questions?

Q1. What is a Hypothesis in machine learning?

In machine learning, a hypothesis is an assumed mapping function between input features & the target output. It represents the model's belief about how inputs relate to outputs, and the learning process involves adjusting parameters to find the hypothesis that best fits the data while generalizing well to unseen cases.

A hypothesis hypothesis is not the same as a model - the hypothesis is the assumption, while the model is the mathematical representation used to test. For example, in linear regression, the hypothesis can be expressed as:

$$\Rightarrow h(x) = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$$

Here, the weights $w_0, w_1, \dots, w_n$ are learned to

minimize a cost function such as mean squared error (MSE).

Hypothesis space (H) is the set of all possible legal hypotheses an algorithm can choose from. The goal of training is to select the single best hypothesis 'h' from 'H' that minimizes prediction error.

Example of Linear Regression Hypothesis

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

```
# Sample data
```

```
x = np.array([1], [2], [3], [4])
```

```
y = np.array([2, 4, 6, 8])
```

```
# Train model (Find best hypothesis)
```

```
model = LinearRegression()
```

```
model.fit(x, y)
```

```
# Hypothesis function:  $y = mx + b$ 
```

```
m = model.coef_[0]
```

```
b = model.intercept_
```

```
print(f"Hypothesis:  $y = \{m: .2f\}x + \{b: .2f\}$ ")
```

```
# End
```

Once a hypothesis is chosen, it's evaluated using metrics like MSE, accuracy, precision, recall or F_1 -score. A good hypothesis should:

- Generalize well to unseen data (avoid overfitting)
- Be simple and interpretable
- Be robust to noise & outliers
- Scale efficiently to large datasets

* Statistical perspective

In statistics, hypothesis are tested using:

- null Hypothesis (H_0): No significant relationship exists.
- Alternative Hypothesis (H_1): A significant relationship exists.

Hypothesis testing involves setting a ~~significance~~ significance level (α), computing a test statistic, and using the p-value to decide whether to reject H_0 .

Q2 ~~Explain~~ Explain the impact of taking the learning rate when can take smaller value of learning rate & in which situation the value can be larger?

In machine learning, the learning rate is a hyperparameter that determines the step size at each iteration of an optimization algorithm (like gradient descent) as it moves toward the minimum of a loss function.

Think of it as the speed at which a model "learns" is the model is an explorer trying to find the lowest point in a valley (the minimum error). The learning rate is the length of each step that takes.

1. The Impact of the Learning Rate:

- * Too High: The model takes giant leaps. It may overshoot the minimum and "bounce" back and forth. These will result on loss curve, the loss increases wildly (non-divergence).

* Too low : The model takes tiny, microscopic steps. It makes progress but is extremely inefficient. The result

* optimal : The model takes steps that are large enough to make progress but small enough to settle precisely. The result on loss curve, smooth descent loss decreases steadily & flattens out at a low point.

2) When to use a smaller learning rate

A smaller value (e.g., 10^{-4} or 10^{-5}) is typically chosen in the following situations :

- Fine-tuning a model : When you are using a pre-trained model (transfer learning), the weights are already "mostly correct." You take small steps to refine them without destroying the patterns the model already knows.

- Large and complex datasets : In deep networks with millions of parameters, high learning rates can easily destabilize the training because one "bad" step can ripple through the entire network.

- Near the end of training : many engineers use learning rate decay. You start fast, then shrink the rate as you get closer to the minimum to "settle" into the exact bottom of the valley.

- Small batch sizes : If you are updating the model after looking at only a few examples (small batch size), the "direction" of the gradient is noisier. A smaller learning rate prevents the model from reacting too strongly to this noise.

3) When to use a larger Learning Rate
A larger value (e.g., 0.1 or 0.01) can be beneficial in these scenarios:

- Beginning of Training: when weights are initialized randomly, the model is very far from solution. Large steps help the model quickly move toward the general "neighbourhood" of the optimal solution.
- Simple models / Datasets: If the loss landscape is smooth and simple (like a perfect bowl), you can take larger steps without fear of overshooting.
- Large Batch Sizes: with more data per update, the "direction" provided by the gradient is more accurate and stable, allowing you to move faster with a higher learning rate.
- Escaping local machine minima: sometimes a model gets stuck in a "small dip" that isn't the true bottom. A temporary "warm-up" or high learning rate can give the model enough momentum to "jump out" and find a better, deeper minimum.

3) What is overfitting, and how can you avoid it?

Overfitting occurs when a model memorizes the training data—including its noise and outliers—too perfectly, causing it to perform poorly on new, unseen data.

How to avoid it:

- Cross-validation & use techniques like k-fold to ensure the model performs well across different subsets of data.
- Early Stopping: pause training as soon as the performance on your validation set starts to decline.
- Regularization (L_1/L_2): Apply mathematical penalties to discourage the model from becoming overly complex.
- Data Augmentation: Artificially increase your dataset size by creating variations of existing data (e.g., flipping or rotating images).
- Dropout: Randomly "turn off" neurons during training in neural networks to prevent them from over-relying on specific features.
- Simplify The Architecture: Reduce the number of layers or parameters if the model is too "powerful" for the amount of data available.

4] Why the use of linear regression is not preferred for a classification task?

While linear regression is excellent for predicting continuous numbers (like house prices), it fails as a classifier for several technical & practical reasons:

1] Extrapolation & Invalid probabilities

Linear regression predicts values along a straight line that can extend to infinity ($-\infty$ to $+\infty$).

In classification, we usually want probabilities between 0 and 1. Linear Regression might predict a value of 1.5 or -0.2, which is mathematically meaningless for a "yes/no" category.

2] Sensitivity to outliers

Linear Regression tries to minimize the distance between the line and all data points. If you add an "outlier" (a data point very far to one side), the entire line will shift to accommodate it. This shift can move the "decision boundary," causing the model to classify points it previously got right.

3] The "Hard" Decision Boundary

Classification assumes a clear "cut-off" point. Linear Regression assumes a ~~constant~~ constant rate of change. If you use threshold (like 0.5) to separate classes, the linear model treats a point at 0.51 and a point at 0.99 as essentially the same class, but it doesn't handle the "S-curve" transition (uncertainty) near the boundary well.

4] Categorical misinterpretation

If you have three classes (e.g., Apple, orange, Banana) and assign them numbers 1, 2, and 3, Linear Regression assumes an order and distance. It would mathematically assume that an orange is "halfway" between an apple and banana, which is not true for distinct categories.

[5] Why do we perform normalization?

In machine learning, models see numbers, not context. If one feature is measured in thousands and another in units, the model will incorrectly assume the larger numbers are more important.

Normalization solves this through three main benefits:

- Fairness: It puts all features on a "level playing field" (e.g., a scale of 0 to 1), so the model treats a change in weight as equally important to a change in height.

- Speed: It makes the math easier for optimization algorithms. Instead of "zig-zagging" through a distorted space, the model can find the best solution in a direct path, saving time and computing power.

- Geometric Accuracy: For models that calculate "distance" (like KNN or K-means), it prevents features with large ranges from completely dominating the distance calculation.

When can be skipped?

Generally don't need to normalize for tree-based models (like Random Forest or XGBoost), as they split data based on thresholds rather than calculating distances or gradients.

[6] What is the bias-variance tradeoff?

It's the mean balance between a model being too simple versus too complex.

○ Bias (underfitting): The model is too rigid. It ignores the patterns in your data & misses the mark because it oversimplifies everything.

□ Analogy: Studying only the table of contents for an exam.

○ Variance (overfitting): The model is too flexible. It memorizes the specific training data - including the random noise - so it can't generalize to new information.

□ Analogy: Memorizing the exact answers to practice test instead of learning the concept.

The Goal

We want the "Sweet Spot" where total error is minimized.

○ Increase complexity $\Rightarrow$ Bias drops, but variance climbs.

○ Decrease complexity $\Rightarrow$ variance drops, but bias climbs.

□ What is "Training Set" and "Test Set" in machine learning model? what is requirement of training, validation, and test sets?

In machine learning, our dataset is typically split into three distinct "buckets." Think of them like ~~the training set~~ the stage of preparing for a high-stakes exam.

1. The Training Set "The Textbook"

This is the data your model actually "looks at" to learn. It provides identifiable patterns, adjusts

its internal weights, and tried to understand the relationship between inputs & outputs.

- o Goal is to teach the model.

- o Size is usually the largest portion (e.g., 60-80% of your data).

2] The validation set (The quiz practice)

This is a "holdout" set that is never touched until the very end. The model only sees this once all training and tuning are finished.

- o Goal is to give an unbiased, real-world estimate of how the model will perform on data it has never encountered before.

- o Requirement is [It must remain strictly isolated. If you use the test set to make and decisions about the model, you have "leaked" information, and your accuracy scores will be false.

Why do we need all these

using only two set (Train & Test) is

like taking the final exam, realizing you failed, and then studying specifically for the question you missed before retaking it. You aren't actually getting smarter; you are just memorizing that specific test.

In Summary:

Training $\Rightarrow$ Build the model's knowledge.

Validation $\Rightarrow$ Choose the best version of the model

Test $\Rightarrow$ Final evaluation of finished model.

[8] How do you handle missing or corrupted data in a dataset?

Handling missing or corrupted data is critical because most machine learning algorithms cannot handle "nan" (Not a Number) or "null" values and will simply crash. The strategy we choose depends on why the data is missing and how much of it is gone.

1. Deletion (The Quick Fix)

If amount of missing data is very small (e.g., less than 5%), we can simply remove it.

- Row Deletion: delete entire row.

- Column Deletion: delete entire column section (only do this if the column is mostly empty or unimportant).

- Risk: we might lose valuable information or introduce bias if the missing data is not random.

2. Imputation (Filling The Gaps)

This is the process of replacing missing values with estimated ones.

- Categorical Data: Replace missing values with the mode (the most frequent category).

- Numerical Data: mean / median:

Replacing with the average. Median is better if you have outliers.

- Constant value: Replacing with "0" or a flag like "missing" to tell the model that the absence of data is itself a signal.

3] Advanced Imputation (predictive)

Instead of using simple averages, we use the other data points to "guess" the missing value.

K-nearest neighbors (KNN) % Find the "most similar" rows and use their values to fill the gap.

Multivariate Imputation (MICE) % A statistical method that models each feature with missing values as a function of other features.

4] Handling Corrupted Data

Corrupted data (like a "999" for age or "NA" in price column) requires data cleaning %

- Outliers detection : Use z-scores or interquartile Range (IQR) % find values that are physically impossible.

- pattern matching % Use regular expressions (Regex) % fix formatting issues (e.g., dates written in different styles).

- constraint checking : If a "discount" column is higher than the "price" column, we know the data is corrupted and needs manual adjustment or deletion.

9] Explain the evaluation matrix for classification problem.

In machine learning, we can not rely on a single number to tell you if a model is "good". Evaluation metrics help you understand exactly where your model is succeeding - and where it's failing.

Most of these metrics start with the confusion matrix.

1 The Foundation : Confusion matrix

The confusion matrix is a table used to describe the performance of a classification model. It compares the predicted labels against the actual labels.

- True positive (TP): you predicted "YES" and it was "YES".
- True negative (TN): you predicted "NO" and it was "NO".
- False positive (FP): you predicted "YES" but it was "NO". (Type I Error).
- False negative (FN): you predicted "NO" but it was actually "YES". (Type II Error).

2 The Core metrics

Once we have the four values above, we can calculate the following:

□ Accuracy : The percentage of total guesses that were correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- When to use : when our classes are balanced (e.g., 50% cats, 50% dogs).

The trap : if 99% of people are healthy, a model that predicts "Healthy" for everyone is 99% accurate but completely useless at finding sick people.

□ Precision (Quality)

at all the time the model predicts "positive," how many were actually positive?

$$\text{precision} = \frac{TP}{TP + FP}$$

• **use case** : When the cost of a false positive is high (e.g., spam detection - you don't want an important work email going to the spam/junk folder).

Recall / Sensitivity (Quantity)

Of all the actual positive cases that exist, how many did the model find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

use case : When the cost of a false negative is high (e.g., cancer detection - missing a sick patient is much worse than a false alarm).

F_1 - Score (The Balance)

The harmonic mean of precision and Recall. It gives you a single score that punishes extreme values.

$$F_1 = 2 \times \frac{\text{precision} \times \text{Recall}}{\text{precision} + \text{Recall}}$$

31 **Advanced** Visualization metrics

ROC curve & AUC

The ROC curve plots the True Positive Rate vs. the False Positive Rate at different classification thresholds.

• **AUC (Area Under the Curve)** : A single number from 0 to 1. An AUC of 1.0 is a perfect model; 0.5 is no better than a random coin flip.

Precision - Recall Curve

Better than the ROC curve when we have highly imbalanced data sets (e.g., detecting credit card fraud where only 0.1% of transactions are fraudulent).

10] Explain The evaluation matrix for regression problem

In regression, we aren't predicting categories (like "cat" vs. "dog"), but rather continuous numerical values (like a house price or a temperature).

Because our prediction will almost never be 100% exact, evaluation metrics focus on measuring error distance - The "gap" between what the model predicted and the actual truth.

1] Mean Absolute Error (MAE)

This is the simplest metric. It takes the average of the absolute differences between the predicted values and the actual values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

• Interpretation: on average, how far off are our predictions?

• pros: It is easy to understand & uses the same units as our target (e.g., "our model is off by \$5000 on average").

• cons: It means treats all errors equally. A \$100 error is just 10x worse than a \$10 error.

2] Mean Squared Error (MSE)

Instead of taking the absolute value, we square the errors before averaging them.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

• Interpretation: The average squared distance.

• pros: It highlights large errors.

Because errors are squared, a large mistake is penalized much more heavily than a small one.

- Cons: This units are squared (e.g., "dollars squared"), which makes it hard to explain to non-technical stakeholders.

3] Root Mean Squared Error (RMSE)

This simplifies the square root of the MSE. It is arguably the most popular metric for regression.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Interpretation: The standard deviation of the residuals (errors).

Pros: Like MSE, it penalizes large errors, but it brings the units back to the original scale (back to "dollars").

4] R-Squared (R^2) - Coefficient of determination

Unlike the others, this isn't an "error distance". It is a score between 0 and 1 that tells you how well your model explains the variance in the data.

- $R^2 = 1$: perfect model.
- $R^2 = 0$: Your model is no better than just guessing the average value every time.
- Interpretation: "My model explains 85% of the variation in house prices."

5] Adjusted R-Squared

Regular R^2 has a flaw: It will always increase (or stay the same) if you add more

features, that don't even if those features are useless. Adjusted R^2 penalizes you for adding features that don't improve the model's predictive power.

Metric	Best used when	Sensitivity to outliers
MSE	we want simple, human-readable errors.	Low (Robust)
MSE	we want to heavily punish large mistakes.	High
RMSE	we want to punish large mistakes but keep units readable.	High
R^2	we want to know how much "info" your model captured	N/A